IT5437 Assignment 1

Intensity Transformations and Neighborhood Filtering

Index No: 249309R

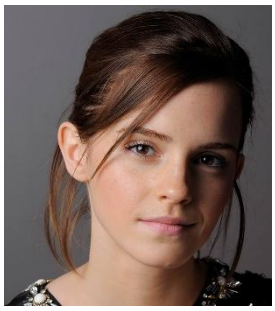
Name: I.D Jayawardana

Github URL - [deshanj/uom-msc-computer-vision-assignment-01 at development]

Question 01

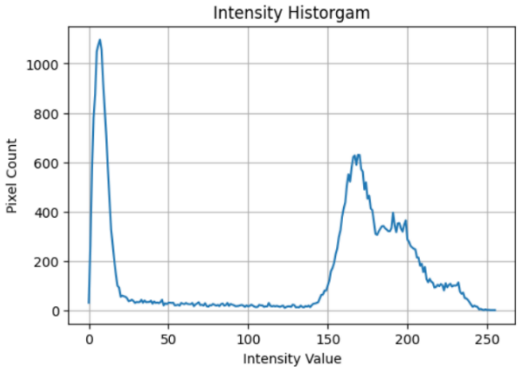
```python
#Question 01

import cv2 as cv
import numpy as np

image_1b = cv.imread('a1images/emma.jpg')

c = np.array([(50,100),(150,255)])

transform = np.arange(256,dtype='uint8')

transform[c[0,0]:c[0,1]+1] = np.linspace(c[1,0],c[1,1],c[0,1]-c[0,0]+1).astype('uint8')

image_1b_out = cv.LUT(image_1b,transform)

cv.imshow('image_1b_out', image_1b_out)
cv.waitKey(0)
cv.destroyAllWindows()
```

✓ 24.2s



Question 02:

First, I implemented an Intensity Histogram with pixel counts, to identify the intensity bands for Grey Matter and White Matter using the peaks.



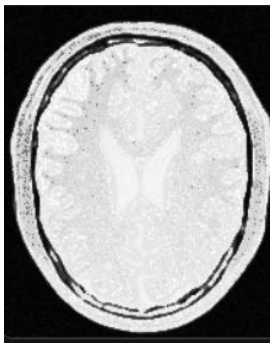
Based on that,

- Gray Matter: 150-190
- White Matter: 190-255

Applied relevant Transformations:

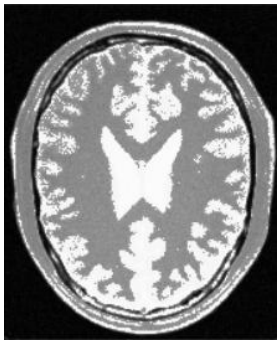
```python
#Grey Matter --> 150-190
import cv2 as cv
import numpy as np

image_2 = cv.imread('a1images/brain_proton_density_slice.png', cv.IMREAD_GRAYSCALE)

c = np.array([(150,190),(200,255)])
ri = 30 # reduced intensity

transform = np.arange(256, dtype='uint8')

transform[0:c[0,0]+1] = np.linspace(0, c[0,0]-ri, c[0,0]+1).astype('uint8')
transform[c[0,0]:c[0,1]+1] = np.linspace(c[1,0], c[1,1], c[0,1]-c[0,0]+1).astype('uint8') # Grey Matter boosted
transform[c[0,1]+1:256] = np.linspace(c[1,1]-ri, 255, 256-(c[0,1]+1)).astype('uint8')

image_2_out_1 = cv.LUT(image_2, transform)

cv.imshow('image_2_out_1', image_2_out_1)
cv.waitKey(0)
cv.destroyAllWindows()
```

✓ 4.0s



Grey Matter

```python
#White Matter --> 190 -255

import cv2 as cv
import numpy as np

image_2 = cv.imread('a1images/brain_proton_density_slice.png', cv.IMREAD_GRAYSCALE)

c = np.array([(190,255),(245,255)])
ri = 30 # reduced intensity

transform = np.arange(256, dtype='uint8')

transform[0:c[0,0]+1] = np.linspace(0, c[0,0]-ri, c[0,0]+1).astype('uint8')
transform[c[0,0]:c[0,1]+1] = np.linspace(c[1,0], c[1,1], c[0,1]-c[0,0]+1).astype('uint8') # White Matter boosted
transform[c[0,1]+1:256] = np.linspace(c[1,1]-ri, 255, 256-(c[0,1]+1)).astype('uint8')

image_2_out_1 = cv.LUT(image_2, transform)

cv.imshow('image_2_out_1', image_2_out_1)
cv.waitKey(0)
cv.destroyAllWindows()
```
✓ 10.3s



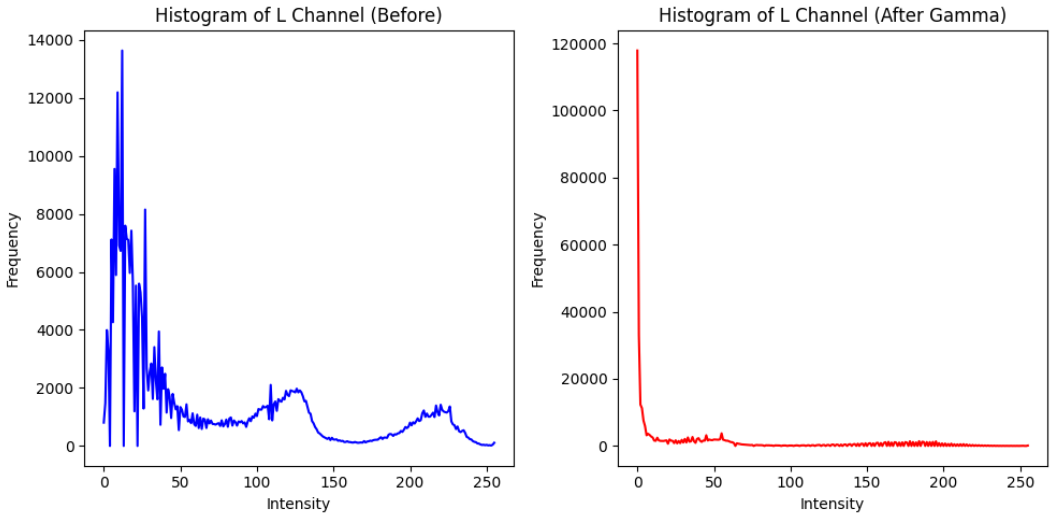
White Matter

Question 03:

```python
import cv2 as cv
import numpy as np

gamma = 2.2

image_3 = cv.imread('a1images/highlights_and_shadows.jpg')
lab = cv.cvtColor(image_3, cv.COLOR_BGR2Lab)
L, a, b = cv.split(lab)

gamma_LUT = np.array([((i / 255.0) ** gamma) * 255 for i in np.arange(256)]).astype("uint8")

L_gamma = cv.LUT(L, gamma_LUT)
lab_gamma = cv.merge([L_gamma, a, b])

image_3_out = cv.cvtColor(lab_gamma, cv.COLOR_Lab2BGR)

cv.imshow("Original", image_3)
cv.imshow("Gamma Corrected", image_3_out)
cv.waitKey(0)
cv.destroyAllWindows()
```
✓ 3.1s

Gamma Corrected

Assumed γ value as 2.2



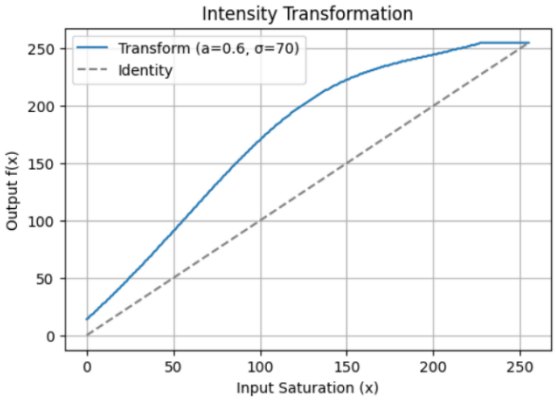
Question 04

```python
import cv2 as cv
import numpy as np

image_4 = cv.imread('a1images/spider.png')

hsv = cv.cvtColor(image_4, cv.COLOR_BGR2HSV)
H, S, V = cv.split(hsv)

sigma = 70
a = 0.6

x = np.arange(256)
transform = x + a * 128 * np.exp(-((x - 128) ** 2) / (2 * sigma ** 2))
transform = np.clip(transform, 0, 255).astype(np.uint8)

S_new = cv.LUT(S, transform)

hsv_new = cv.merge([H, S_new, V])
image_vibrant = cv.cvtColor(hsv_new, cv.COLOR_HSV2BGR)

cv.imshow("Original", image_4)
cv.imshow("Vibrance Enhanced", image_vibrant)

plt.figure(figsize=(6,4))
plt.plot(x, transform, label=f"Transform (a={a}, σ={sigma})")
plt.plot(x, x, '--', color='gray', label="Identity")
plt.xlabel("Input Saturation (x)")
plt.ylabel("Output f(x)")
plt.title("Saturation Intensity Transformation")
plt.legend()
plt.grid(True)
plt.show()

cv.waitKey(0)
cv.destroyAllWindows()
```

After multiple testing, value for “a” was taken as 0.6, since it provided a visually pleasing output.

Original

Vibrance Enhanced

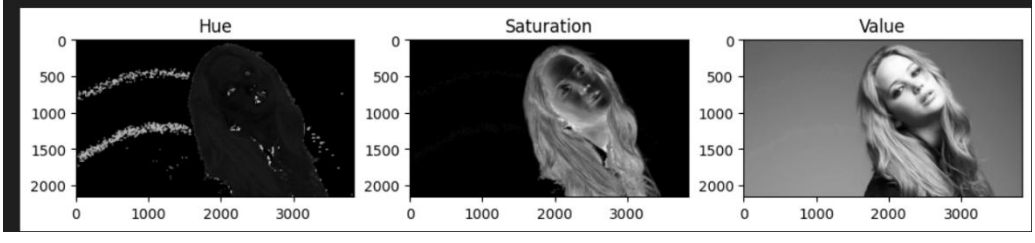


Question 05

(a)

```python
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread("a1images/jeniffer.jpg")
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
h, s, v = cv.split(hsv)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(h, cmap='gray'); plt.title("Hue")
plt.subplot(1, 3, 2); plt.imshow(s, cmap='gray'); plt.title("Saturation")
plt.subplot(1, 3, 3); plt.imshow(v, cmap='gray'); plt.title("Value")
plt.show()
```

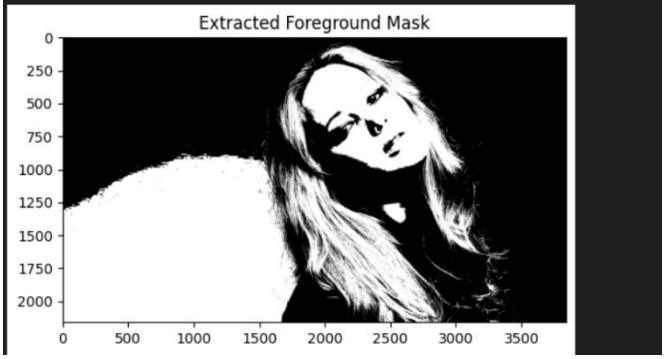
✓ 3.2s



(b)

```
_, mask = cv.threshold(v, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

plt.imshow(mask, cmap='gray'); plt.title("Extracted Foreground Mask"); plt.show()
```

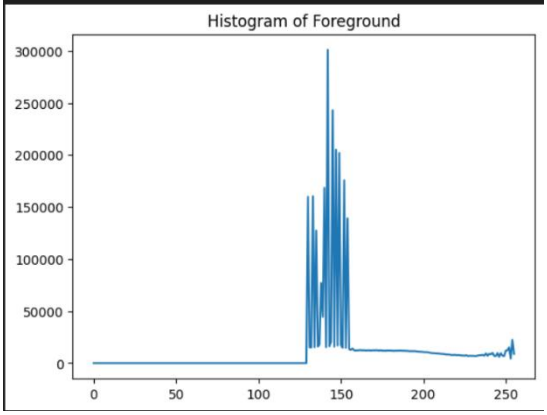
✓ 0.8s



(c)

```
foreground = cv.bitwise_and(v, v, mask=mask)
hist = cv.calcHist([foreground], [0], mask, [256], [0,256])
plt.plot(hist); plt.title("Histogram of Foreground"); plt.show()
```

✓ 0.0s



(d) & (e)

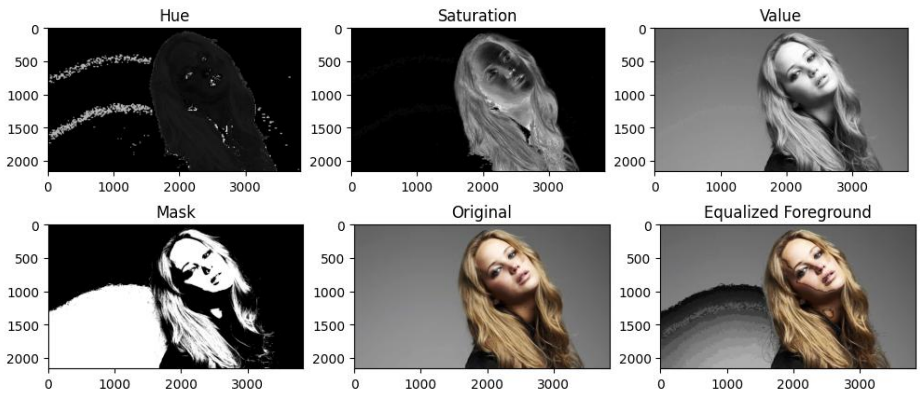
```
L = 256                     # Number of intensity levels
MN = np.sum(mask > 0)       # Total number of pixels in foreground

cdf = hist.cumsum()
cdf_normalized = cdf / MN

t = np.array([(L - 1) * cdf_normalized[k] for k in range(L)], dtype=np.uint8)

equalized_foreground = t[foreground]
```

0.0s

(f)

```
background = cv.bitwise_and(v, v, mask=cv.bitwise_not(mask))
result_v = cv.add(background, equalized_foreground)


result_hsv = cv.merge([h, s, result_v])
result_img = cv.cvtColor(result_hsv, cv.COLOR_HSV2BGR)


plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1); plt.imshow(h, cmap='gray'); plt.title("Hue")
plt.subplot(2, 3, 2); plt.imshow(s, cmap='gray'); plt.title("Saturation")
plt.subplot(2, 3, 3); plt.imshow(v, cmap='gray'); plt.title("Value")
plt.subplot(2, 3, 4); plt.imshow(mask, cmap='gray'); plt.title("Mask")
plt.subplot(2, 3, 5); plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); plt.title("Original")
plt.subplot(2, 3, 6); plt.imshow(cv.cvtColor(result_img, cv.COLOR_BGR2RGB)); plt.title("Equalized Foreground")
plt.show()
```

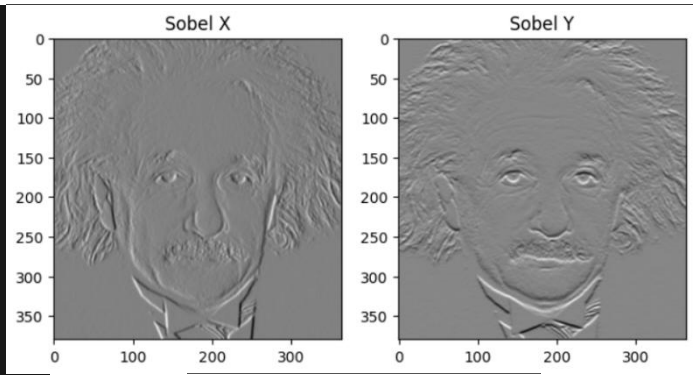
3.9s

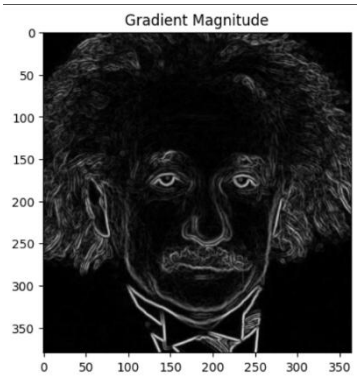


Question 06

(a)

```python
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt


image_6 = cv.imread("a1images/einstein.png", cv.IMREAD_GRAYSCALE)

# Sobel kernels
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]], dtype=np.float32)

sobel_y = np.array([[-1, -2, -1],
                    [ 0,  0,  0],
                    [ 1,  2,  1]], dtype=np.float32)

# Apply filter2D
grad_x = cv.filter2D(image_6, cv.CV_64F, sobel_x)
grad_y = cv.filter2D(image_6, cv.CV_64F, sobel_y)

# Compute gradient magnitude
grad_mag = np.sqrt(grad_x**2 + grad_y**2)
grad_mag = np.uint8(255 * grad_mag / np.max(grad_mag))


plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 2); plt.imshow(grad_x, cmap='gray'); plt.title("Sobel X")
plt.subplot(1, 3, 3); plt.imshow(grad_y, cmap='gray'); plt.title("Sobel Y")
plt.show()

plt.imshow(grad_mag, cmap='gray'); plt.title("Gradient Magnitude"); plt.show()
```





(b)

```python
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]], dtype=np.float32)

sobel_y = np.array([[-1, -2, -1],
                    [ 0,  0,  0],
                    [ 1,  2,  1]], dtype=np.float32)

def convolve(image, kernel):
    h, w = image.shape
    kh, kw = kernel.shape
    pad = kh // 2
    padded = np.pad(image, pad, mode='constant')
    result = np.zeros_like(image, dtype=np.float32)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+kh, j:j+kw]
            result[i, j] = np.sum(region * kernel)
    return result

grad_x = convolve(image_6, sobel_x)
grad_y = convolve(image_6, sobel_y)

grad_mag = np.sqrt(grad_x**2 + grad_y**2)
grad_mag = np.uint8(255 * grad_mag / np.max(grad_mag))

plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1); plt.imshow(image_6, cmap='gray'); plt.title("Original")
plt.subplot(1, 3, 2); plt.imshow(grad_x, cmap='gray'); plt.title("Manual Sobel X")
plt.subplot(1, 3, 3); plt.imshow(grad_y, cmap='gray'); plt.title("Manual Sobel Y")
plt.show()
plt.imshow(grad_mag, cmap='gray'); plt.title("Gradient Magnitude (Manual)")
plt.show()
```

(c)

```python
kx = np.array([1, 0, -1], dtype=np.float32).reshape(1, 3)   # horizontal
ky = np.array([1, 2, 1], dtype=np.float32).reshape(3, 1)    # vertical
 Ctrl+L to chat, Ctrl+K to generate
intermediate = cv.filter2D(image_6, cv.CV_32F, kx)

grad_x_sep = cv.filter2D(intermediate, cv.CV_32F, ky)

kx_y = np.array([1, 2, 1], dtype=np.float32).reshape(1, 3)
ky_y = np.array([1, 0, -1], dtype=np.float32).reshape(3, 1)

intermediate_y = cv.filter2D(image_6, cv.CV_32F, kx_y)
grad_y_sep = cv.filter2D(intermediate_y, cv.CV_32F, ky_y)

grad_mag = np.sqrt(grad_x_sep**2 + grad_y_sep**2)
grad_mag = np.uint8(255 * grad_mag / np.max(grad_mag))

plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1); plt.imshow(image_6, cmap='gray'); plt.title("Original")
plt.subplot(1, 3, 2); plt.imshow(grad_x_sep, cmap='gray'); plt.title("Sobel X (Separable)")
plt.subplot(1, 3, 3); plt.imshow(grad_y_sep, cmap='gray'); plt.title("Sobel Y (Separable)")
plt.show()

plt.imshow(grad_mag, cmap='gray'); plt.title("Gradient Magnitude (Separable)")
plt.show()
 0.3s
```

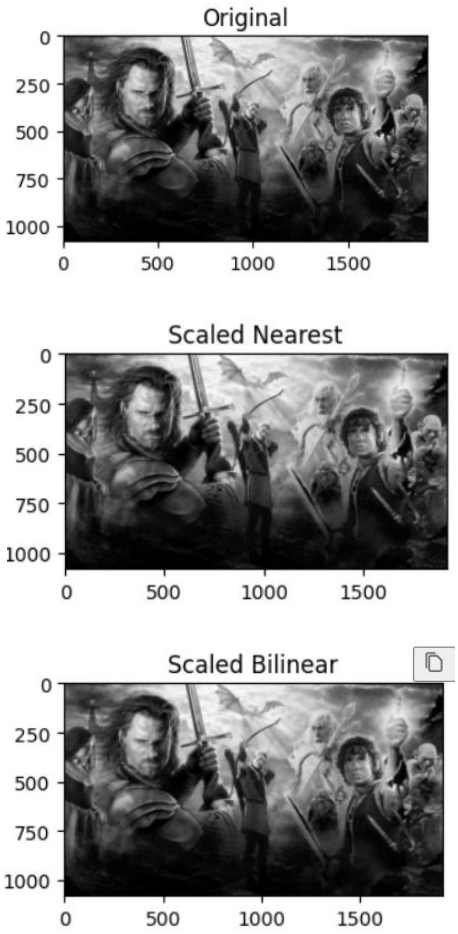
Question 07

```python
def zoom_image(img, factor, method="nearest"):
    h, w = img.shape[:2]
    new_h, new_w = int(h * factor), int(w * factor)
    result = np.zeros((new_h, new_w), dtype=img.dtype)

    if method == "nearest":
        for i in range(new_h):
            for j in range(new_w):
                src_x = int(i / factor)
                src_y = int(j / factor)
                result[i, j] = img[src_x, src_y]

    elif method == "bilinear":
        for i in range(new_h):
            for j in range(new_w):
                x = i / factor
                y = j / factor

                x0, y0 = int(np.floor(x)), int(np.floor(y))
                x1, y1 = min(x0+1, h-1), min(y0+1, w-1)

                dx, dy = x - x0, y - y0

                val = (1-dx)*(1-dy)*img[x0, y0] + dx*(1-dy)*img[x1, y0] + \
                      (1-dx)*dy*img[x0, y1] + dx*dy*img[x1, y1]

                result[i, j] = int(val)

    return result
```

✓ 0.0s

```python
def normalized_ssd(img1, img2):
    diff = (img1.astype(np.float32) - img2.astype(np.float32)) ** 2
    ssd = np.sum(diff)
    norm = np.sum(img1.astype(np.float32) ** 2)
    return ssd / norm
```

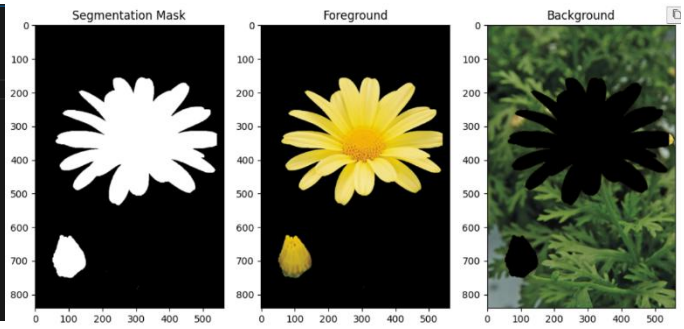
✓ 0.0s



Question 08

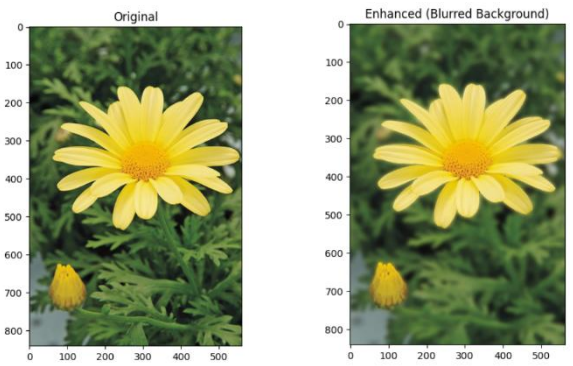
(a)

```python
image_7 = cv.imread("a1images/daisy.jpg")
mask = np.zeros(image_7.shape[:2], np.uint8)

bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)

rect = (20, 20, image_7.shape[1]-40, image_7.shape[0]-40)

cv.grabCut(image_7, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)

mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')

foreground = image_7 * mask2[:, :, np.newaxis]
background = image_7 * (1 - mask2[:, :, np.newaxis])
```



(b)

```python
blurred_bg = cv.GaussianBlur(image_7, (31, 31), 0)

enhanced = blurred_bg.copy()
enhanced[mask2 == 1] = image_7[mask2 == 1]
```



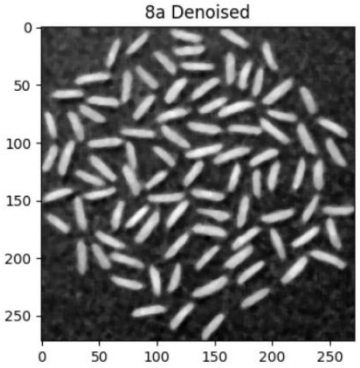
(c) GrabCut produces a binary mask where the pixel boundary between the flower and background is uncertain.

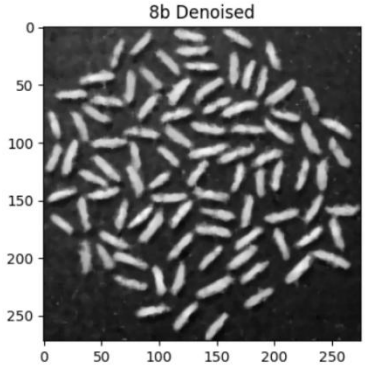
This boundary pixels mostly get classified as background (0) instead of foreground (1). So, when the blurred background is inserted, these “mixed” pixels contribute to dark edges around the edges of the flower.

Question 09

(a), (b)

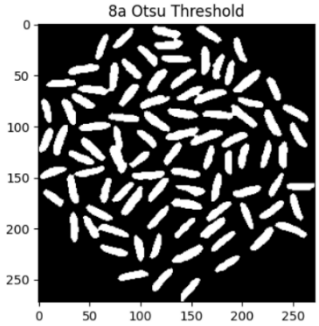
```
image_8a = cv.imread('a1images/rice_8a.png', cv.IMREAD_GRAYSCALE)
image_8b = cv.imread('a1images/rice_8b.png', cv.IMREAD_GRAYSCALE)

image_8a_denoised = cv.medianBlur(image_8a, 5)
image_8b_denoised = cv.medianBlur(image_8b, 5)
```

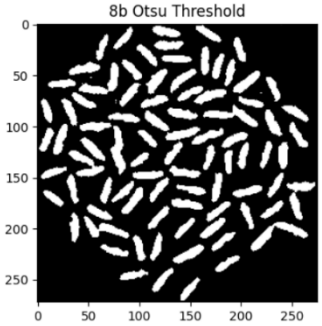




(c)

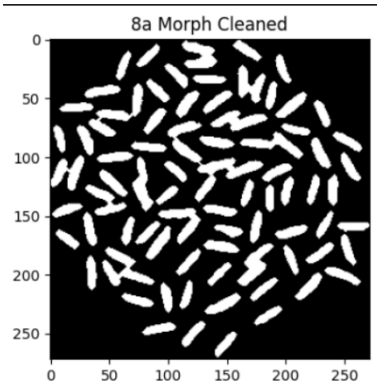
```
_, threshold_8a = cv.threshold(image_8a_denoised, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)
_, threshold_8b = cv.threshold(image_8b_denoised, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)
```

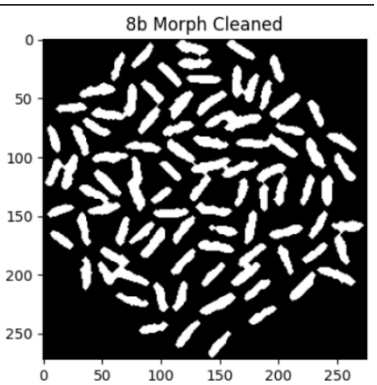




(d)

```
kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (3,3))

clean_8a = cv.morphologyEx(thresh_8a, cv.MORPH_OPEN, kernel, iterations=2)
clean_8b = cv.morphologyEx(thresh_8b, cv.MORPH_OPEN, kernel, iterations=2)

clean_8a = cv.morphologyEx(clean_8a, cv.MORPH_CLOSE, kernel, iterations=2)
clean_8b = cv.morphologyEx(clean_8b, cv.MORPH_CLOSE, kernel, iterations=2)
```





(e) Subtracted 1 because the background is also counted as one.

```
num_labels_8a, labels_8a = cv.connectedComponents(clean_8a)
num_labels_8b, labels_8b = cv.connectedComponents(clean_8b)

print(f"Number of rice grains in 8a: {num_labels_8a - 1}")
print(f"Number of rice grains in 8b: {num_labels_8b - 1}")
```

✓ 0.0s

```
Number of rice grains in 8a: 75
Number of rice grains in 8b: 75
```

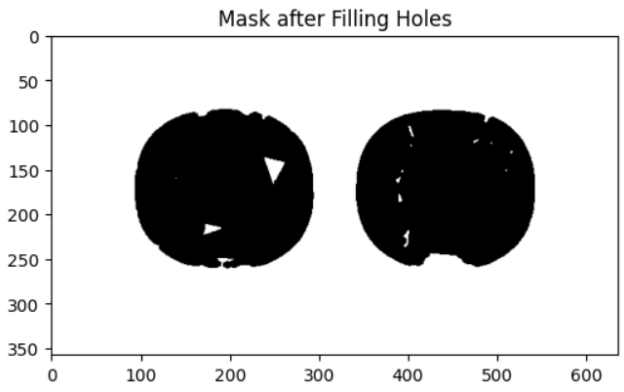
Question 10

(a), (b), (c)

```
sapphire_img = cv.imread('a1images/sapphire.jpg', cv.IMREAD_GRAYSCALE)

blur = cv.GaussianBlur(sapphire_img, (5,5), 0)

_, binary_mask = cv.threshold(blur, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (5,5))

mask_filled = cv.morphologyEx(binary_mask, cv.MORPH_CLOSE, kernel, iterations=2)

plt.figure(figsize=(6,6))
plt.imshow(mask_filled, cmap='gray')
plt.title("Mask after Filling Holes")
plt.show()

num_labels, labels, stats, centroids = cv.connectedComponentsWithStats(mask_filled)

areas_pixels = stats[1:, cv.CC_STAT_AREA]  # skip background

for i, area in enumerate(areas_pixels, 1):
    print(f"Sapphire {i} area in pixels: {area}")
```

✓ 0.0s

```
Sapphire 1 area in pixels: 168854
Sapphire 2 area in pixels: 29
Sapphire 3 area in pixels: 12
Sapphire 4 area in pixels: 3
Sapphire 5 area in pixels: 3
Sapphire 6 area in pixels: 6
Sapphire 7 area in pixels: 330
Sapphire 8 area in pixels: 5
Sapphire 9 area in pixels: 3
Sapphire 10 area in pixels: 19
Sapphire 11 area in pixels: 1
Sapphire 12 area in pixels: 25
Sapphire 13 area in pixels: 126
Sapphire 14 area in pixels: 58
```



(d)

```
sensor_width_mm = 36
image_width_px = sapphire_img.shape[1]
px_size = sensor_width_mm / image_width_px

f = 8
d = 480

M = f / d

areas_real_mm2 = areas_pixels * (px_size / M)**2

for i, area_mm2 in enumerate(areas_real_mm2, 1):
    print(f"Sapphire {i} actual area: {area_mm2:.2f} mm^2")
```

✓ 0.0s

```
Sapphire 1 actual area: 1947621.79 mm^2
Sapphire 2 actual area: 334.50 mm^2
Sapphire 3 actual area: 138.41 mm^2
Sapphire 4 actual area: 34.60 mm^2
Sapphire 5 actual area: 34.60 mm^2
Sapphire 6 actual area: 69.21 mm^2
Sapphire 7 actual area: 3806.34 mm^2
Sapphire 8 actual area: 57.67 mm^2
Sapphire 9 actual area: 34.60 mm^2
Sapphire 10 actual area: 219.15 mm^2
Sapphire 11 actual area: 11.53 mm^2
Sapphire 12 actual area: 288.36 mm^2
Sapphire 13 actual area: 1453.33 mm^2
Sapphire 14 actual area: 668.99 mm^2
```